

文章笔记

OpenAI Codex 最佳实践

基于公开课程资料整理

2025

作者/来源: OpenAI

发布日期: 2025

文章链接: <https://developers.openai.com/codex/learn/best-practices>

目录

1 引言	3
1.1 本章小结	3
2 Prompt 编写与上下文管理	3
2.1 结构化 Prompt 的四要素	3
2.2 Reasoning Level 选择	3
2.3 复杂任务先规划再执行	4
2.4 本章小结	4
3 AGENTS.md: 持久化指导	4
3.1 什么是 AGENTS.md	4
3.2 多级 AGENTS.md 体系	4
3.3 编写原则	5
3.4 本章小结	5
4 系统配置	5
4.1 config.toml 配置体系	5
4.2 Sandbox 与 Approval 机制	6
4.3 本章小结	6
5 测试与代码审查	6
5.1 闭环验证流程	6
5.2 /review 命令	6
5.3 本章小结	7
6 MCP: 外部工具集成	7
6.1 什么是 MCP	7
6.2 MCP 的配置方式	7
6.3 本章小结	7
7 Skills: 技能封装	8
7.1 什么是 Skill	8
7.2 适用场景	8
7.3 本章小结	8
8 Automations: 自动化调度	9
8.1 从 Skill 到 Automation	9
8.2 适用场景	9
8.3 本章小结	9
9 Session 管理与常用命令	10
9.1 Session 的本质	10
9.2 常用 Slash 命令	10
9.3 本章小结	10

10 常见错误	10
10.1 本章小结	11
11 总结与延伸	11
11.1 拓展阅读	11

1 引言

OpenAI Codex 是一款 coding agent，支持 CLI、IDE 扩展和 Codex App 三种使用方式。本文档是 OpenAI 官方发布的 Codex 最佳实践指南，覆盖了从 prompting、规划、配置到 MCP 集成、Skills 封装和自动化调度的完整 workflow。

核心理念

不要把 Codex 当成一次性的助手，而应当视其为一个**需要持续配置和改进的队友**。核心路径：
正确的任务上下文 → AGENTS.md 持久化指导 → 配置标准化 → MCP 外部集成 → Skills 封装 → Automations 自动化。

1.1 本章小结

本文围绕六大主题展开：Prompt 编写、AGENTS.md 配置、系统配置、MCP 外部工具集成、Skills 技能封装、以及 Automations 自动化，旨在帮助用户从“能用”走向“好用”。

2 Prompt 编写与上下文管理

2.1 结构化 Prompt 的四要素

Codex 即使在 prompt 不够完善时也能给出不错的结果，但在大型代码库或高风险任务中，结构化的 prompt 能显著提升可靠性。官方推荐在每个 prompt 中包含以下四个要素：

Prompt 四要素

1. **Goal** (目标)：你想要修改或构建什么？
2. **Context** (上下文)：哪些文件、目录、文档、示例或错误与本任务相关？可用 @ 引用文件。
3. **Constraints** (约束)：需要遵守哪些标准、架构要求、安全规则或编码规范？
4. **Done when** (完成条件)：任务完成时应当满足什么条件？例如测试通过、行为变更或 bug 不再复现。

这四个要素帮助 Codex 限定作用范围、减少假设，并产出更容易 review 的代码。

2.2 Reasoning Level 选择

Codex 支持多个推理等级，适配不同任务复杂度：

推理等级	适用场景
Low	边界清晰的快速任务
Medium / High	较复杂的代码变更或调试
Extra High	长链路、高推理需求的 agent 任务

表 1: Reasoning Level 与任务复杂度的对应关系

语音输入加速上下文提供

在 Codex App 中，可以使用语音听写 (speech dictation) 来口述任务需求，而不是手动打字。这对需要提供大量上下文的场景尤其高效。

2.3 复杂任务先规划再执行

对于复杂、模糊或难以描述的任务，应当让 Codex 先做规划 (planning)，再进入实现阶段。官方推荐三种方式：

1. **Plan mode**: 通过 `/plan` 或 `Shift+Tab` 切换。Codex 会先收集上下文、提出澄清问题，再制定实施方案。这是大多数用户最简单有效的选择。
2. **让 Codex 采访你**: 当你有模糊的想法但不知如何表述时，让 Codex 主动提问、挑战你的假设，把模糊想法转化为具体方案。
3. **PLANS.md 模板**: 对于更高级的工作流，可配置 Codex 遵循 `PLANS.md` 或 `execution-plan` 模板，适用于长期运行或多步骤工作。

常见错误：跳过规划

在多步骤和复杂任务中跳过规划 (planning) 是最常见的错误之一。直接开始编码往往导致方向偏差和大量返工。

2.4 本章小结

好的 prompt 包含目标、上下文、约束和完成条件四个要素。根据任务复杂度选择合适的 reasoning level。对于复杂任务，使用 Plan mode 或让 Codex 先采访你，避免直接进入实现。

3 AGENTS.md：持久化指导

3.1 什么是 AGENTS.md

`AGENTS.md` 是一个开放格式的文件，可以理解为给 agent 看的 README。它会被自动加载到上下文中，是编码团队标准化 Codex 行为的最佳位置。

AGENTS.md 应覆盖的内容

- 仓库布局 and 重要目录
- 项目运行方式
- Build、test、lint 命令
- 工程规范和 PR 期望
- 约束和”不要做”规则
- ”完成”的定义及验证方式

3.2 多级 AGENTS.md 体系

Codex 支持在多个层级放置 `AGENTS.md`，形成层级覆盖机制：

- **全局级**: `~/.codex/AGENTS.md`, 存放个人默认偏好
- **仓库级**: 项目根目录的 `AGENTS.md`, 存放团队共享标准
- **子目录级**: 特定目录中的 `AGENTS.md`, 存放局部规则

就近优先原则

当多个层级存在 `AGENTS.md` 时, 离当前工作目录更近的文件优先级更高。这类似于 `.gitignore` 的覆盖逻辑。

3.3 编写原则

- **保持简洁**: 短而准确的 `AGENTS.md` 比冗长模糊的规则有用得多
- **按需添加**: 先写基础内容, 只在发现重复性错误后才添加新规则
- **拆分引用**: 当文件过大时, 保持主文件精简, 并引用任务级别的 markdown 文件(如 `code_review.md`、`architecture.md`)
- **回顾驱动**: 当 Codex 犯同样的错误两次时, 让它做 `retrospective` 并更新 `AGENTS.md`

常见错误: 在 Prompt 中堆积持久性规则

把本应写在 `AGENTS.md` 中的持久性规则反复写在 `prompt` 里, 是使用 Codex 时最常见的反模式之一。应当将经过验证的 `prompting` 模式迁移到 `AGENTS.md` 中。

CLI 中可以使用 `/init` 命令快速生成一份 starter `AGENTS.md`。

3.4 本章小结

`AGENTS.md` 是 Codex 持久化指导的核心机制。通过多级文件体系实现全局、仓库和目录级别的规则覆盖。保持文件简洁、按需迭代, 并利用 `retrospective` 机制持续改进。

4 系统配置

4.1 config.toml 配置体系

Codex 的配置通过 `config.toml` 文件管理, 支持两个层级:

- **个人默认**: `~/.codex/config.toml` (通过 Codex App 的 `Settings` → `Configuration` → `Open config.toml` 打开)
- **仓库特定**: `.codex/config.toml`, 存放项目级别的行为配置
- **命令行覆盖**: 仅用于一次性场景 (CLI 用户)

可配置项包括: `model` 选择、`reasoning effort`、`sandbox mode`、`approval policy`、`profiles`、`MCP setup` 等。

4.2 Sandbox 与 Approval 机制

Codex 内置操作系统级别的沙箱机制，提供两个关键控制旋钮：

两个安全控制旋钮

- **Approval mode**：决定 Codex 何时需要请求你的许可才能执行命令
- **Sandbox mode**：决定 Codex 能否在目录中读写，以及可以访问哪些文件

新手建议：先紧后松

如果你刚接触 coding agent，应当保持默认的严格权限。只在了解 workflow、确认信任特定仓库后，才逐步放宽权限。不要一开始就给 Codex 完全的系统权限。

CLI、IDE 和 App 共享配置

CLI、IDE 扩展和 Codex App 共享同一套配置层级。在任一端修改配置都会影响其他端的行为。

4.3 本章小结

通过 `config.toml` 的个人级和仓库级配置实现一致性行为。Sandbox 和 Approval 是两个核心安全旋钮，建议新手从严格权限开始，逐步放宽。

5 测试与代码审查

5.1 闭环验证流程

不要仅仅让 Codex 做代码变更就结束。应当要求它完成完整的验证闭环：

1. 为变更编写或更新测试
2. 运行相关的测试套件
3. 检查 lint、格式化或类型检查
4. 确认最终行为与需求一致
5. 检查 diff 中是否存在 bug、回归或风险模式

关键前提：让 Codex 知道什么是”好”

Codex 能够执行这个验证闭环，但前提是它知道什么算”好”。这些信息可以来自 prompt 或 `AGENTS.md`，包括测试命令、lint 配置、代码风格期望等。

5.2 `/review` 命令

`/review` 是一个内置的代码审查命令，支持多种审查模式：

- 对比 base branch 进行 PR 风格审查
- 审查未提交的变更
- 审查某个特定 commit

- 使用自定义审查指令

如果团队有 `code_review.md` 文件并在 `AGENTS.md` 中引用，Codex 在审查时也会遵循该指导。

GitHub Cloud 集成

如果使用 GitHub Cloud，可以设置 Codex 自动审查 PR。在 OpenAI 内部，Codex 审查了 100% 的 PR。可以启用自动审查，也可以在需要时通过 `@Codex` 触发审查。

5.3 本章小结

Codex 不仅能生成代码，还能测试、检查和审查代码。通过 `/review` 命令和 `code_review.md` 配置，实现团队级别一致的代码审查标准。在 GitHub Cloud 上还可以实现全自动 PR 审查。

6 MCP：外部工具集成

6.1 什么是 MCP

Model Context Protocol (MCP) 是一个开放标准，用于将 Codex 连接到外部工具和系统。当 Codex 所需的上下文不在代码仓库中时，MCP 就是解决方案。

何时使用 MCP

- 所需上下文在仓库之外（如 issue tracker、monitoring 系统）
- 数据频繁变化（如实时指标、日志）
- 希望 Codex 使用工具而非依赖粘贴的指令
- 需要跨用户、跨项目的可复用集成

6.2 MCP 的配置方式

Codex 支持两种 MCP server 类型：

- **STDIO**：标准输入/输出通信
- **Streamable HTTP**：支持 OAuth 认证的 HTTP 通信

配置入口：

- Codex App：Settings → MCP servers
- CLI：使用 `codex mcp add` 命令添加自定义 server

不要一次性接入所有工具

只在工具确实能解锁某个真实工作流时才添加。不要一开始就把所有常用工具都接入。从 1-2 个明确能消除手动重复操作的工具开始，然后逐步扩展。

6.3 本章小结

MCP 是 Codex 连接外部系统的标准协议，支持 STDIO 和 Streamable HTTP 两种模式。按需接入、从小处开始是正确的使用策略。

7 Skills：技能封装

7.1 什么是 Skill

当某个工作流变得可重复，就不应继续依赖长 prompt 或反复的交互。Skill 将指令（SKILL.md 文件）、上下文和支持逻辑封装为一个可跨 CLI、IDE 和 App 一致调用的单元。

Skill 设计原则

- 每个 skill 聚焦于一个任务
- 从 2-3 个具体用例出发
- 定义清晰的输入和输出
- 描述（description）要说明 skill 做什么、何时使用
- 包含用户实际会说的触发短语

7.2 适用场景

Skills 特别适合以下类型的重复性工作：

- 日志分类（Log triage）
- Release notes 起草
- 按 checklist 进行 PR 审查
- 迁移规划（Migration planning）
- 遥测或事件摘要（Telemetry / incident summaries）
- 标准调试流程

Skill 的存储位置

- 个人 skills: \$HOME/.agents/skills/
 - 团队共享 skills: 仓库内的 .agents/skills/ 目录
- 团队共享的 skills 对新成员的 onboarding 尤其有帮助。

可以使用内置的 \$skill-creator skill 来快速生成第一版 skill。建议在本地迭代稳定后，再打包为 plugin 进行更广泛的分享。

经验法则

如果你发现自己反复使用同一个 prompt，或反复纠正同一个工作流，它就应该成为一个 skill。

7.3 本章小结

Skill 是将验证过的工作流封装为可复用单元的机制。聚焦单一任务、从具体用例出发、先本地迭代后再分享。

8 Automations: 自动化调度

8.1 从 Skill 到 Automation

当一个工作流已经稳定可靠，就可以用 Automation 让 Codex 在后台按计划自动执行。

Skills vs Automations

Skills 定义方法，Automations 定义调度。如果一个工作流仍然需要大量人工干预，应当先将其封装为 skill。只有当它足够可预测时，automation 才能真正发挥乘数效应。

8.2 适用场景

好的 automation 候选包括：

- 汇总近期 commits
- 扫描潜在 bug
- 起草 release notes
- 检查 CI 失败
- 生成 standup 摘要
- 按计划运行重复性分析

在 Codex App 的 Automations 标签页中，可以选择：

- 运行的项目
- 执行的 prompt（可调用 skills）
- 运行的节奏/频率
- 执行环境（dedicated git worktree 或本地环境）

先手动验证，再自动化

在工作流手动运行已经可靠之前，不要将其转为 automation。过早自动化不可靠的任务只会放大错误。

用 Automation 做反思和维护

Automation 不仅仅用于执行任务。还可以用来审查近期 session、汇总重复性摩擦、改进 prompt 和工作流设置。

8.3 本章小结

Automation 是将稳定 skill 按计划自动执行的机制。先确保手动运行可靠，再自动化。可用于执行，也可用于反思和维护。

9 Session 管理与常用命令

9.1 Session 的本质

Codex 的 session 不只是聊天记录，而是随时间累积上下文、决策和操作的工作线程。良好的 session 管理对输出质量有重要影响。

9.2 常用 Slash 命令

命令	功能
/plan	切换到规划模式
/review	代码审查
/init	生成 starter AGENTS.md
/experimental	切换实验性功能
/resume	恢复保存的会话
/fork	创建新线程并保留原始记录
/compact	压缩/总结较早的上下文
/agent	切换并行 agent 线程
/theme	选择语法高亮主题
/apps	在 Codex 中使用 ChatGPT apps
/status	查看当前 session 状态

表 2: Codex 常用 Slash 命令一览

一任务一线程

保持一个线程对应一个连贯的工作单元。只有当工作真正产生分支时才使用 `/fork`。使用一个线程管理整个项目（而非单个任务）会导致上下文膨胀，降低输出质量。

Subagent 工作流

可以使用 Codex 的 subagent 功能将有限的工作从主线程卸载出去。主 agent 保持聚焦于核心问题，subagent 处理探索、测试或分类等辅助任务。

9.3 本章小结

Session 是积累上下文的工作线程。通过 slash 命令管理线程、压缩上下文、分叉和切换 agent。坚持“一任务一线程”的原则，使用 subagent 卸载辅助工作。

10 常见错误

以下是官方总结的使用 Codex 时最常见的错误：

八大常见错误

1. 在 prompt 中堆积持久性规则，而不是迁移到 AGENTS.md 或 skill
2. 没有告诉 agent 如何运行 build 和 test 命令，导致它看不到自己工作的结果
3. 在多步骤复杂任务中跳过规划
4. 在理解工作流之前就给 Codex 完全的系统权限
5. 在同一批文件上运行多个活跃线程，而不使用 git worktree
6. 在工作流手动运行尚不可靠时就将其转为 automation
7. 把 Codex 当成需要逐步看管的工具，而不是与自己并行工作的队友
8. 使用一个线程管理整个项目而非每个任务一个线程，导致上下文膨胀

10.1 本章小结

避免上述八大常见错误，核心原则是：持久化规则写入 AGENTS.md、复杂任务先规划、权限先紧后松、并行而非看管、一任务一线程。

11 总结与延伸

本文系统梳理了 OpenAI Codex 的最佳实践，核心思路可以用一条渐进路径总结：

Codex 使用的渐进路径

结构化 Prompt → AGENTS.md 持久化 → config.toml 标准化 → MCP 外部集成 → Skills 封装 → Automations 调度

几个值得深入关注的方向：

- **AGENTS.md 的团队协作模式**：如何在多人团队中维护和演进 AGENTS.md，使其成为活的工程规范文档
- **MCP 生态**：随着 MCP 作为开放标准的发展，更多第三方工具将可通过 MCP 接入 Codex
- **Skill 的 Plugin 化**：从个人本地 skill 到团队共享 plugin 的路径，是提升组织效能的关键
- **Automation 的反思性用法**：不仅用 automation 执行重复任务，还可用于定期回顾和改进工作流本身

11.1 拓展阅读

- OpenAI Codex 官方文档：<https://developers.openai.com/codex>
- Codex Prompting Guide：<https://developers.openai.com/codex/learn>
- Model Context Protocol (MCP) 规范：<https://modelcontextprotocol.io>
- Codex 配置参考：<https://developers.openai.com/codex/docs/configuration>